

1. Understand Why NullPointerException Happens (Interview Notes)

What is NullPointerException?

A **NullPointerException (NPE)** occurs when you try to **access a method, field, or property using a reference that is null**.

Example:

```
String name = null;  
  
System.out.println(name.length());
```

Output

```
NullPointerException
```

Because name does **not refer to any object**.

Why Does It Happen?

A reference variable can either:

- Point to an object 
- Be null 

```
String name = null;
```

Trying to use it:

```
name.length();
```

The JVM cannot find an object to call the method on, so it throws a **NullPointerException**.

Common Causes

- Calling a method on a null object.
 - Accessing a field of a null object.
 - Accessing an array through a null reference.
 - Returning null from a method without checking it.
-

Important Interview Points

- NullPointerException is an **unchecked exception** (RuntimeException).
 - It occurs when a reference is null, **not** when an object is empty.
 - The exception happens **at runtime**, not compile time.
 - Initializing an object before using it prevents most NPEs.
 - Modern Java encourages using Optional to reduce unnecessary null handling.
-

Tricky Interview Questions

Q1. Is NullPointerException checked or unchecked?

Answer:

Unchecked (RuntimeException).

Q2. Does an empty String cause NullPointerException?

```
String s = "";
```

Answer: No.

An empty string is still a valid object.

Q3. Which throws NullPointerException?

```
String s = null;  
  
s.length();
```

Answer:

Calling length() because s is null.

Q4. Is null an object?

Answer: No.

It represents the **absence of an object reference**.

Interview Summary

- **NullPointerException occurs when a null reference is used like an object.**
- It is one of the most common runtime exceptions in Java.
- It is prevented by proper initialization, null checks, and using Optional where appropriate.

Interview One-Liner:

"**NullPointerException occurs when code tries to access an object through a reference that is null.**"

2. Use Null Checks When Input Can Be Absent (Interview Notes)

Why Perform Null Checks?

If a method parameter may be null, check it **before using it**.

Without a check:

```
public void printName(String name) {  
    System.out.println(name.length());  
}
```

```
}
```

Passing null causes an exception.

Better Approach

```
public void printName(String name) {  
    if (name != null) {  
        System.out.println(name.length());  
    }  
}
```

Now the method safely handles missing input.

When to Use Null Checks?

- User input
 - Database values
 - API responses
 - Method parameters
 - External libraries
-

Important Interview Points

- Always validate inputs that may be null.
- Null checks prevent NullPointerException.
- Validate as early as possible ("fail fast").
- If null is not allowed, consider using `Objects.requireNonNull()` to fail with a clear message.

Example:

```
Objects.requireNonNull(name, "Name cannot be null");
```

Tricky Interview Questions

Q1. Why should input be validated?

Answer:

To prevent runtime exceptions and improve reliability.

Q2. Which is safer?

```
if(name != null)
```

or

```
name.length();
```

Answer:

`if(name != null).`

Q3. Is a null check always required?

Answer:

No.

Only when the value can legitimately be null.

Q4. What utility method checks for null?

Answer:

`Objects.requireNonNull().`

Interview Summary

- **Perform null checks whenever input may be missing.**
- Validate data before accessing methods or fields.
- Fail fast for invalid input instead of allowing hidden errors later.

- Use `Objects.requireNonNull()` when null is unacceptable.

Interview One-Liner:

"Validate nullable inputs before using them to prevent `NullPointerException` and improve code reliability."

3. Use Optional as a Return Type for Possibly Missing Values (Interview Notes)

What is Optional?

Optional is a container object that **may or may not contain a value**.

Instead of returning null, a method can return an Optional.

Example

Without Optional:

```
public User findUser() {  
    return null;  
}
```

With Optional:

```
public Optional findUser() {  
    return Optional.empty();  
}
```

The caller now knows the value may be absent.

Why Use Optional?

It makes the possibility of a missing value **explicit**, reducing accidental `NullPointerException`s.

Important Interview Points

- Optional represents **a value that may or may not exist**.
 - It is mainly used as a **method return type**.
 - It encourages callers to handle missing values explicitly.
 - Do **not** use Optional for fields, setters, or method parameters in typical Java code.
 - Optional improves API readability and reduces reliance on null.
-

Tricky Interview Questions

Q1. Why use Optional instead of returning null?

Answer:

It clearly communicates that a value may be absent and encourages explicit handling.

Q2. Should Optional be used as a field?

Answer: Generally, no.

It is primarily intended for **return types**.

Q3. Is Optional a replacement for all null checks?

Answer: No.

It mainly improves method return values, not every use of null.

Q4. Where is Optional commonly used?

Answer:

Repository/service methods (e.g., Spring Data JPA's `findById()`).

Interview Summary

- **Optional** represents an optional return value instead of returning null.
- It makes APIs clearer and reduces the chance of NullPointerException.
- Best used as a **return type**, especially in repository and service methods.

Interview One-Liner:

"Optional explicitly represents a value that may be absent, making APIs safer than returning null."

4. Use Optional.of(), ofNullable(), empty(), isPresent(), ifPresent() (Interview Notes)

Common Optional Methods

Optional.of()

Creates an Optional containing a **non-null** value.

```
Optional name = Optional.of("Java");
```

✗ Passing null throws NullPointerException.

Optional.ofNullable()

Creates an Optional that accepts **both null and non-null** values.

```
Optional name = Optional.ofNullable(value);
```

If value is null, it returns an empty Optional.

Optional.empty()

Creates an empty Optional.


```
Optional name = Optional.empty();
```

isPresent()

Checks whether a value exists.

```
if (name.isPresent()) {  
    System.out.println(name.get());  
}
```

ifPresent()

Executes code only if a value exists.

```
name.ifPresent(System.out::println);
```

Important Interview Points

- Optional.of() requires a **non-null** value.
 - Optional.ofNullable() safely handles both null and non-null values.
 - Optional.empty() represents no value.
 - isPresent() checks if a value exists.
 - ifPresent() executes code only when a value is available.
 - Prefer ifPresent(), orElse(), or orElseThrow() over calling get() directly.
-

Tricky Interview Questions

Q1. Difference between of() and ofNullable()?

Answer:

- of() → Does **not** allow null.
 - ofNullable() → Accepts null and returns an empty Optional.
-

Q2. What happens here?

```
Optional.of(null);
```

Answer:

Throws NullPointerException.

Q3. Which method creates an empty Optional?

Answer:

```
Optional.empty();
```

Q4. Which is preferred?

```
optional.get();
```

or

```
optional.ifPresent(...);
```

Answer:

ifPresent() (or orElse(), orElseThrow()) because get() can throw NoSuchElementException if the Optional is empty.

Q5. Does isPresent() return the value?

Answer: No.

It only returns true or false.

Interview Summary

- **Optional provides utility methods to safely work with values that may be absent.**
- `of()` requires a non-null value, while `ofNullable()` accepts null.
- `empty()` represents no value.
- `isPresent()` checks for existence, and `ifPresent()` executes logic only when a value exists.
- Modern Java favors Optional methods over direct null checks for return values.

Interview One-Liner:

"Optional.of() requires a value, ofNullable() safely handles null, and methods like ifPresent() help avoid explicit null checks and runtime errors."